

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

Lucrarea de laborator Nr. 4

Disciplina: Securitatea informațională

Tema: Exerciții interactive de programare în Python

A efectuat: st. gr. SI-264

Radu Sologub

A verificat: **Reișman Patricia**, asis. univ.

Chișinău, 2026

Lucrarea de laborator nr. 4

Introducere — sarcina și mediul de lucru

Lucrarea urmărește parcurgerea a 22 de exerciții de pe platforma [LearnPython.org](https://learnpython.org): 13 din secțiunea *Learn the Basics* și primele 9 din *Advanced Tutorials*. Temele încep cu afișarea unui mesaj și tipurile de date, apoi trec prin colecții, controlul execuției, funcții, obiecte, module și tehnici avansate de prelucrare a datelor.

Exercițiile au fost efectuate în editorul `script.py` al platformei. Acesta afișează rezultatele în panoul *IPython Shell* și verifică soluția la trimitere. Capturile din 28 septembrie 2026 documentează enunțurile, codul introdus și rezultatele vizibile. Toate cele 22 de teme din listă au fost parcurse.

1. Desfășurarea lucrării

Pentru fiecare temă s-a completat exercițiul în editorul platformei și s-a observat rezultatul din panoul *IPython Shell*. Capturile sunt așezate imediat lângă rezultatul pe care îl documentează.

Instrucțiuni, tipuri și șiruri

run

Exercise

Use the "print" function to print the line "Hello, World!".

script.py

```
1 print("Hello, World!")
```

IPython Shell

```
<script.py> output:
  Goodbye, World!

<script.py> output:
  Hello, World!

In [1]: |
```

Great job!

Solution

Submit

Partial fu

Code Inti

Closures

Decoratc

Map, Filt

Parsing C

DataCarr

This site is generously supported by [DataCamp](https://datacamp.com). DataCamp offers online interactive [Python Tutorials](https://datacamp.com/python-tutorials) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.1 - **Hello, World!.** Programul afișează mesajul Hello, World!.

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

script.py	IPython Shell
<pre>1 # change this code 2 mystring = "hello" 3 myfloat = 10.0 4 myint = 20 5 6 # testing code 7- if mystring == "hello": 8 print("String: %s" % mystring) 9- if isinstance(myfloat, float) and myfloat == 10.0: 10 print("Float: %f" % myfloat) 11- if isinstance(myint, int) and myint == 20: 12 print("Integer: %d" % myint)</pre>	<pre><script.py> output: String: hello <script.py> output: String: hello Float: 10.000000 Integer: 20 In [1]: </pre>

Great job!

[Solution](#) [Submit](#) ●

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.2 - Variables and Types. Sunt afișate șirul hello, numărul real 10.000000 și întregul 20.

You will also have to fill in the variable `second_name` with the second name in the `names` list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py	IPython Shell
<pre>1 numbers = [] 2 strings = [] 3 names = ["John", "Eric", "Jessica"] 4 5 # write your code here 6 second_name = None 7 numbers.append(1) 8 numbers.append(2) 9 numbers.append(3) 10 strings.append("hello") 11 strings.append("world") 12 13 second_name = names[1] 14 15 # this code should write out the filled arrays 16 # and the second name in the names list (Eric).</pre>	<pre><script.py> output: [1, 2, 3] ['hello', 'world'] The second name on the names list is Eric In [1]: </pre>

Great Job!

[Solution](#) [Submit](#) ●

[Argument](#)
[Regular Expressions](#)
[Exceptions](#)
[Sets](#)
[Serialization](#)
[Partial Function Application](#)
[Code Interactions](#)
[Closures](#)
[Decorators](#)
[Map, Filter, Reduce](#)
[Parsing with Regular Expressions](#)
[DataCamp](#)

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.3 - Lists. Listele sunt completate, iar al doilea nume este Eric.

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

script.py	IPython Shell
<pre>1 x = object() 2 y = object() 3 4 # TODO: change this code 5 x_list = [x] * 10 6 y_list = [y] * 10 7 big_list = x_list + y_list 8 9 print("x_list contains %d objects" % len(x_list)) 10 print("y_list contains %d objects" % len(y_list)) 11 print("big_list contains %d objects" % len(big_list)) 12 13 # testing code 14 if x_list.count(x) == 10 and y_list.count(y) == 10:</pre>	<pre><script.py> output: x_list contains 10 objects y_list contains 10 objects big_list contains 20 objects Almost there... Great! In [1]:</pre>
Good work!	
Solution	Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.4 - Basic Operators. Două liste de câte 10 elemente sunt concatenate într-o listă de 20.

Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.`

script.py	IPython Shell
<pre>1 data = ("John", "Doe", 53.44) 2 format_string = 'Hello %s %s. Your current 3 balance is %s.' 4 print(format_string % data)</pre>	<pre><script.py> output: Hello John Doe. Your current balance is 53.440000 <script.py> output: Hello John Doe. Your current balance is \$53.440000 <script.py> output: Hello John Doe. Your current balance is \$53.44 <script.py> output: Hello John Doe. Your current balance is \$53.44 <script.py> output: Hello John Doe. Your current balance is \$53.44.</pre>
Great work!	
Solution	Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.5 - String Formatting. Mesajul final afișează soldul \$53.44 cu două zecimale.

Try to fix the code to print out the correct information by changing the string.

script.py	IPython Shell
<pre>18 # Convert everything to uppercase 19 print("String in uppercase: %s" % s.upper()) 20 21 # Convert everything to lowercase 22 print("String in lowercase: %s" % s.lower()) 23 24 # Check how a string starts 25 if s.startswith("Str"): 26 print("String starts with 'Str'. Good!") 27 28 # Check how a string ends 29 if s.endswith("ome!"): 30 print("String ends with 'ome!'. Good!") 31 32 # Split the string into three separate strings, 33 # each containing only a word 34 print("Split the words of the string: %s" % s .split(" "))</pre>	<pre><script.py> output: Length of s = 20 The first occurrence of the letter a = 8 a occurs 2 times The first five characters are 'Strx ' The next five characters are 'xxxxaa' The thirteenth character is 'x' The characters with odd index are 'txxxax xm!' The last five characters are 'xome!' String in uppercase: STRX XXXAAXXX XXOME! String in lowercase: strx xxxaaxxx xxome! String starts with 'Str'. Good! String ends with 'ome!'. Good! Split the words of the string: ['Strx', 'xxxxaaxx', , 'xome!']</pre>

Solution Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.6 - Basic String Operations. Sunt afișate lungimea, căutarea, decuparea, schimbarea literelor și separarea șirului.

Controlul execuției și organizarea codului

Indexarea listelor începe de la zero, iar o condiție sau o buclă poate selecta doar valorile cerute. Exercițiile următoare extind aceste idei la funcții, obiecte și module.

Run

Exercise

Change the variables in the first section, so that each if statement resolves as True.

script.py	IPython Shell
<pre>1 # change this code 2 number = 16 3 second_number = 0 4 first_array = [1, 3, 4] 5 second_array = [1,2] 6 7 if number > 15: 8 print("1") 9 10 if first_array: 11 print("2") 12 13 if len(second_array) == 2: 14 print("3") 15 16 if len(first_array) + len(second_array) == 5: 17 print("4")</pre>	<pre><script.py> output: 1 2 3 4 5 <script.py> output: 1 2 3 4 5 6</pre>

Great Work!

Solution Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.7 - Conditions. Valorile alese activează ramurile condiționale cerute.

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

```
1 219, 390, 984, 592, 236, 105, 942, 941,
2
3 4 386, 462, 47, 418, 907, 344, 236, 375, 823,
4 566, 597, 978, 328, 615, 953, 345,
5 399, 162, 758, 219, 918, 237, 412, 566, 826,
6 248, 866, 950, 626, 949, 687, 217,
7 815, 67, 104, 58, 512, 24, 892, 894, 767,
8 553, 81, 379, 843, 831, 445, 742, 717,
9 958, 609, 842, 451, 688, 753, 854, 685, 93,
10 857, 440, 380, 126, 721, 328, 753, 470,
11 743, 527
12 ]
13
14 # your code goes here
15 for number in [n for n in numbers if n % 2 == 0
16 or n == 237]:
17
18
```

IPython Shell

```
236
942
386
462
418
344
236
566
978
328
162
758
918
237
```

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.8 - Loops. Numerele pare sunt afișate în ordinea listei, până la valoarea de oprire 237.

code , Easier code reuse , Allowing programmers to share and connect code together

2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
1 # Modify this function to return a list of
2 strings as defined above
3 def list_benefits():
4     return [ "More organized code", "More
5     readable code", "Easier code reuse", "Allowing
6     programmers to share and connect code together"]
7
8 # Modify this function to concatenate to each
9 benefit - " is a benefit of functions!"
10 def build_sentence(benefit):
11     return benefit + " is a benefit of
12     functions!"
13
14 def name_the_benefits_of_functions():
15     list_of_benefits = list_benefits()
16     for benefit in list_of_benefits:
17         print(build_sentence(benefit))
```

IPython Shell

```
<script.py> output:
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code
together is a benefit of functions!

In [1]: |
```

Nice work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.9 - Functions. Funcțiile construiesc patru propoziții despre beneficiile lor.

Run

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth \$60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth \$10,000.00.

script.py

```
5 color = ""
6 value = 100.00
7 def description(self):
8     desc_str = "%s is a %s %s worth $%.2f."
9     % (self.name, self.color, self.kind, self.value)
10    return desc_str
11 # your code goes here
12 car1 = Vehicle()
13 car2 = Vehicle()
14
15 car1.name = "Fer"
16 car1.color = "red"
17 car1.value = 60000.0
18
19 car2.name = "Jump"
```

IPython Shell

```
<script.py> output:
  Fer is a red car worth $60000.00.
  Jump is a blue car worth $10000.00.

In [1]: |
```

Great job!

Solution

Submit

1.10 - **Classes and Objects.** Obiectele Fer și Jump sunt descrise prin nume, culoare, tip și valoare.

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
1 phonebook = {
2     "John" : 938477566,
3     "Jack" : 938377264,
4
5     "Jake" : 938273443
6 }
7 # your code goes here
8
9 # testing code
10 if "Jake" in phonebook:
11     print("Jake is listed in the phonebook.")
12
13 if "Jill" not in phonebook:
14     print("Jill is not listed in the phonebook.")
15 )
```

IPython Shell

```
<script.py> output:
  Jake is listed in the phonebook.

<script.py> output:
  Jake is listed in the phonebook.
  Jill is not listed in the phonebook.

In [1]: |
```

Nice work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.11 - **Dictionaries.** Jake este prezent în agenda telefonică, iar Jill a fost eliminată.

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py

```
1 import re
2
3 # Your code goes here
4 find_members = [x for x in sorted(dir(re)) if
  "find" in x]
```

IPython Shell

In [1]: |

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.12 - **Modules and Packages.** Sunt selectate alfabetice numele din modulul `re` care conțin `find`.

În exercițiul de intrare standard, codul pentru citirea și însumarea a trei numere a fost introdus. În sesiunea documentată, executorul LearnPython.org nu a furnizat linia de intrare solicitată de `input()` și a răspuns cu `EOF when reading a line`. Figura 1.13 arată codul și eroarea platformei împreună.

Exercise

Complete the code below so that it reads three integers from a single line, converts each to an `int`, and prints their sum.

script.py

```
1 # write your code here
2 s = sum([int(x) for x in input().split()])
3 print("sum is %s" % s)
4
```

IPython Shell

DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line

In [1]: |

Solution

Submit

Restart Session

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.13 - **Input and Output.** Soluția introdusă citește și însumează trei valori; executorul site-ului returnează `EOF` la citire.

Tehnici avansate

Ultimele exerciții folosesc generatoare, comprehensiuni, expresii regulate și transformări ale funcțiilor sau ale datelor.

will simultaneously switch the values of a and b.

script.py

```

1 # fill in this function
2 def fib():
3     yield 1
4     yield 2
5     a, b = 1, 2
6     while True:
7         tmp = a + b
8         a = b
9         b = tmp
10        yield b
11
12
13 # testing code
14 import types
15 if type(fib()) == types.GeneratorType:
16     print("Good, The fib function is a generator")

```

IPython Shell

```

<script.py> output:
Good, The fib function is a generator.
1
2
3
5
8
13
21
34
55
89

```

Good work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.14 - Generators. Generatorul produce valorile Fibonacci de la 1 până la 89.

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```

1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6, 12.7]
2 newlist = [int(x) for x in numbers if x >= 0]
3 print(newlist)

```

IPython Shell

```

<script.py> output:
[34, 44, 68, 44, 12]

In [1]:

```

Very nice!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.15 - List Comprehensions. Lista rezultată este [34, 44, 68, 44, 12].

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1 l = [2,4,7,3,14,19]
2 for i in l:
3     print(i % 2 == 1)
```

IPython Shell

```
Even
Even
Odd
Odd
Even
Odd

<script.py> output:
False
False
True
True
False
True
```

Nice work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.16 - **Lambda functions.** Testul de paritate afișează False, False, True, True, False, True.

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
implementation
2 def foo(a, b, c, *adds):
3     return len(adds)
4
5 def bar(a, b, c, **o):
6     return "magicnumber" in o and o.get(
7         "magicnumber") == 7
8
9 # test code
10 if foo(1, 2, 3, 4) == 1:
11     print("Good.")
12 if foo(1, 2, 3, 4, 5) == 2:
13     print("Better.")
14 if bar(1, 2, 3, magicnumber=6) == False:
15     print("Great.")
```

IPython Shell

```
File "script.py", line 6
    "magicnumber" in o && o.get("magicnumber") == 7
                        ^
SyntaxError: invalid syntax

<script.py> output:
Good.
Better.
Great.
Awesome!

In [1]: |
```

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a**

1.17 - **Multiple Function Arguments.** Testele afișează Good., Better., Great. și Awesome!.

As an example of a "proper" email-matching regex (like the one in the exercise), see [this](#)

script.py

```
11 def test_email(your_pattern):
12     pattern = re.compile(your_pattern)
13     emails = ["john@example.com", "python
-list@python.org", "what.t.ian?ug{}ly@email
.com"]
14     for email in emails:
15         if not re.match(pattern, email):
16             print("You failed to match %s" %
(email))
17         elif not your_pattern:
18             print("Forgot to enter a pattern!")
19         else:
20             print("Pass")
21     pattern = r".+\.*" # Your pattern here!
22     test_email(pattern)
23
```

IPython Shell

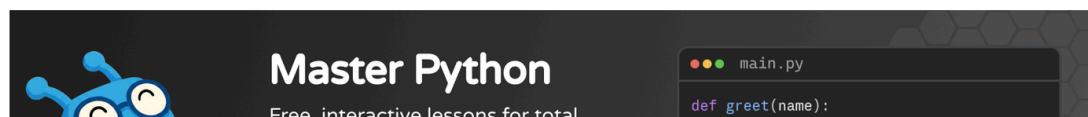
```
<_sre.SRE_Match object; span=(35, 39), match
='[on]'\>
None
Pass
Pass
Pass
<script.py> output:
<_sre.SRE_Match object; span=(35, 39), match
='[on]'\>
None
Pass
Pass
Pass
Pass
```

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.18 - **Regular Expressions.** Exemplele verificate de pagină afișează Pass.

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1 # Setup
2 actor = {"name": "John Cleese", "rank":
"awesome"}
3
4 # Function to modify!!!
5 def get_last_name():
6     return actor["name"].split(' ')[1]
7
8 # Test code
9 get_last_name()
10 print("All exceptions caught! Good job!")
11 print("The actor's last name is %s" %
get_last_name())
```

IPython Shell

```
<script.py> output:
All exceptions caught! Good job!
The actor's last name is Cleese
In [1]: |
```

Great work!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.19 - **Exception Handling.** Este afișat numele de familie Cleese și mesajul de final al testului.

[Hello, W](#)
[Variable](#)
[Lists](#)
[Basic Oj](#)
[String Fi](#)
[Basic St](#)
[Operatic](#)
[Conditio](#)
[Loops](#)
[Function](#)
[Classes](#)
[Dictiona](#)
[Module:](#)
[Package](#)
[Input an](#)
[Generat](#)
[List Cor](#)
[Lambda](#)
[Multiple](#)
[Argume](#)
[Regular](#)
[Exceptio](#)
[Sets](#)
[Serializ](#)

[Closure:](#)
[Decorat](#)
[Map, Fil](#)
[Parsing](#)
[DataCar](#)

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py	IPython Shell
<pre>1 a = ["Jake", "John", "Eric"] 2 b = ["John", "Jill"] 3 print(set(a) - set(b))</pre>	<pre><script.py> output: {'Eric', 'Jake'} In [1]: </pre>

Nice work!

[Solution](#) [Submit](#) ●

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.20 - **Sets**. Diferența dintre participanții celor două evenimente este {Eric, Jake}.

[Run](#) ●

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

script.py	IPython Shell
<pre>1 import json 2 3 # fix this function, so it adds the given name 4 # and salary pair to salaries_json, and return 5 # it 6 def add_employee(salaries_json, name, salary): 7 # Add your code here 8 j = json.loads(salaries_json) 9 j["Me"] = 800 10 salaries_json = json.dumps(j) 11 return salaries_json 12 13 # test code 14 salaries = '{"Alfred" : 300, "Jane" : 400 }' 15 new_salaries = add_employee(salaries, "Me", 800) 16 decoded_salaries = json.loads(new_salaries) 17 print(decoded_salaries["Alfred"])</pre>	<pre><script.py> output: 300 400 800 In [1]: </pre>

Great work!

[Solution](#) [Submit](#) ●

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

1.21 - **Serialization**. Prelucrarea datelor JSON afișează 300, 400 și 800.

Edit the function provided by calling `partial()` and replacing the first three variables in `func()`. Then print with the new partial function using only one input variable so that the output equals 60.

script.py

```

1 #Following is the exercise, function provided:
2 from functools import partial
3 def func(u, v, w, x):
4     return u*4 + v*3 + w*2 + x
5 #Enter your code here to create and print with
  your partial function
6 f = partial(func, 1, 2, 3)
7 print(f(44))

```

IPython Shell

```

<script.py> output:
    20

<script.py> output:
    60

In [1]: |

```

Good job!

Solution **Submit**

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!



1.22 - **Partial functions**. Apelul funcției parțiale produce valoarea cerută, 60.

Concluzii

Au fost parcurse cele 22 de teme Python, de la instrucțiuni elementare până la generatoare, comprehensiuni, expresii regulate, serializare și aplicare parțială. Capturile documentează codul și rezultatele obținute în interfața platformei. Pentru exercițiul de intrare standard, executorul online a întrerupt rularea cu EOF la citirea datelor, deși soluția era introdusă în editor; această limitare a platformei este documentată în captură.

Bibliografie

1. [LearnPython.org](#), [Free Interactive Python Tutorial](#), capitolele *Learn the Basics* și *Advanced Tutorials*, consultat la 28 septembrie 2026.